

# helixqa\_integration\_sec02

## 2. Phase 1: Submodule Dependency Resolution

The first phase of integrating HelixQA into HelixCode is a mechanical prerequisite: every external module that HelixQA imports must be reachable from HelixCode’s working tree, registered in its `.gitmodules` manifest, and locked to a commit that is known to compile and pass tests with the HelixQA revision being introduced. This chapter defines the complete dependency graph discovered by source analysis, the exact `.gitmodules` entries and Git commands required to register them, the cascade bump that brings Catalogizer’s stale HelixQA pin forward, and the Makefile, `go.mod`, and Docker Compose changes that wire the new submodule into the build and runtime environment. All URLs use the SSH transport per the constitutional mandate (Universal Mandatory Constraints §1); no HTTPS URLs appear in any configuration file.

### 2.1 HelixQA Dependency Map

HelixQA is not a standalone binary. Its `go.mod` declares module `digital.vasic.helixqa`<sup>[1]</sup> and imports six sibling modules that live in separate repositories. Some of those siblings have their own transitive dependencies, and HelixQA additionally contains an internal `tools/opensource/` directory with more than 25 vendored open-source packages that carry independent license obligations. The integration plan must account for all three layers.

#### 2.1.1 Direct Dependencies

Analysis of `HelixQA/go.mod` (commit `0bca023`, retrieved 2026-04-30)<sup>[1]</sup> reveals six direct `require` entries that are not satisfiable from the public Go module proxy. Each is resolved inside the Catalogizer workspace via a `replace` directive that points to a sibling checkout:

| Go Module Path                          | Submodule Path             | Repository URL                                                |
|-----------------------------------------|----------------------------|---------------------------------------------------------------|
| <code>digital.vasic.challenges</code>   | <code>../Challenges</code> | <code>git@github.com:vasic-digital/Challenges.git</code>      |
| <code>digital.vasic.containers</code>   | <code>../Containers</code> | <code>git@github.com:vasic-digital/Containers.git</code>      |
| <code>digital.vasic.docprocessor</code> | <code>DocProcessor/</code> | <code>git@github.com:HelixDevelopment/DocProcessor.git</code> |

| Go Module Path                             | Submodule Path                | Repository URL                                                 |
|--------------------------------------------|-------------------------------|----------------------------------------------------------------|
| <code>digital.vasic.llmorchestrator</code> | <code>LLMOrrhestrator/</code> | <code>git@github.com:HelixDevel<br/>LLMOrrhestrator.git</code> |
| <code>digital.vasic.security</code>        | <code>../Security</code>      | <code>git@github.com:vasic-digi<br/>Security.git</code>        |
| <code>digital.vasic.visionengine</code>    | <code>VisionEngine/</code>    | <code>git@github.com:HelixDevel<br/>VisionEngine.git</code>    |

The `digital.vasic.challenges` and `digital.vasic.containers` modules originate from the `vasic-digital` GitHub organization and are already present in the Catalogizer workspace (Challenges at commit 4390e48, containers at 9f9f52a)<sup>[8]</sup>. The remaining four modules—`DocProcessor`, `LLMOrrhestrator`, `Security`, and `VisionEngine`—come from the `HelixDevelopment` organization. Catalogizer already contains `DocProcessor`, `LLMOrrhestrator`, and `VisionEngine` as registered submodules, but `HelixCode` does not. The `Security` module is shared infrastructure and is already present in any workspace that has Catalogizer’s full submodule set.

A critical observation concerns the path convention. `HelixQA`’s `go.mod` expects these modules at sibling paths (`../Challenges`, `../Containers`). Inside `HelixCode` the submodule layout will place `HelixQA` at `HelixQA/` and its dependencies at `Dependencies/` `HelixDevelopment/` to avoid polluting the repository root. This means the `replace` directives in `HelixCode`’s `go.mod` must map to the actual checkout paths, not the relative paths encoded in `HelixQA`’s own `go.mod`.

### 2.1.2 Autonomous Session Dependencies

`HelixQA`’s autonomous QA mode (the 4-phase pipeline in `pkg/autonomous/[9]`) consumes four external modules at runtime. Two of them overlap with the direct Go module dependencies above; two are additional services that communicate via HTTP or gRPC rather than Go import:

| Service                      | Integration Mechanism                                              | Repository                                                      |
|------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>LLMsVerifier</code>    | HTTP REST API                                                      | <code>git@github.com:HelixDevelo<br/>LLMsVerifier.git</code>    |
| <code>LLMOrrhestrator</code> | Go module import<br>( <code>digital.vasic.llmorchestrator</code> ) | <code>git@github.com:HelixDevelo<br/>LLMOrrhestrator.git</code> |
| <code>VisionEngine</code>    | Go module import<br>( <code>digital.vasic.visionengine</code> )    | <code>git@github.com:HelixDevelo<br/>VisionEngine.git</code>    |
| <code>DocProcessor</code>    |                                                                    |                                                                 |

| Service | Integration Mechanism                                           | Repository                                                    |
|---------|-----------------------------------------------------------------|---------------------------------------------------------------|
|         | Go module import<br>( <code>digital.vasic.docprocessor</code> ) | <code>git@github.com:HelixDevelopment/DocProcessor.git</code> |

LLMsVerifier is the outlier. It does not appear in HelixQA’s `go.mod` because it is accessed as a REST service (port 9090 by default)<sup>[10]</sup>. HelixCode already maintains an internal verifier client at `HelixCode/internal/verifier/`<sup>[11]</sup>, but the LLMsVerifier *service* itself is a separate binary that must be built from its own repository. For HelixQA integration, LLMsVerifier must be added as a submodule so that the `helixqa-runner` Docker service (§2.4.4) can compile and start it alongside the main application stack.

### 2.1.3 tools/opensource/ Submodules and License Audit

HelixQA vendors more than 25 open-source tools under `tools/opensource/`. These are not Go modules; they are standalone binaries, model weights, or platform-specific libraries that HelixQA shells out to at runtime. The directory includes packages such as `chromedp`, `scrcpy`, `ffmpeg`, `ollama`, and various perceptual-similarity models<sup>[12]</sup>. Each carries its own license (Apache-2.0, MIT, GPL-3.0, BSD-3-Clause, and others).

Before HelixCode can redistribute or deploy HelixQA in a Docker image, a license audit must verify that no GPL-3.0 dependency is linked statically into a non-GPL binary without an exception. The audit procedure is:

1. Enumerate every package in `tools/opensource/` with its commit hash and declared license.
2. Cross-reference against the SPDX database for license classification.
3. Flag any GPL-2.0-or-later, GPL-3.0, or AGPL package that is compiled into the same process space as proprietary HelixCode code.
4. Document the finding in `docs/audits/helixqa-oss-license-audit.md`.

This audit is tracked as a Phase 1 gate: the submodule may be registered before the audit completes, but the Docker image build (§2.4.4) is blocked until the audit report is signed off.

## 2.2 Submodule Registration in HelixCode

HelixCode’s existing `.gitmodules` (commit `e1307bd`, retrieved 2026-01-18) contains four meta-submodules: `Github-Pages-Website`, `awesome-ai-memory`, `Example_Projects`, and `Example_Resources`<sup>[13]</sup>. None of the HelixQA dependency graph is present. This section defines the exact `.gitmodules` entries to add.

### 2.2.1 Add HelixQA Root Submodule

HelixQA is registered at the repository root so that its relative `replace` directives (`../Challenges`, `../Containers`) resolve correctly when Go builds from inside the `HelixQA/` directory.

```
[submodule "HelixQA"]
  path = HelixQA
  url = git@github.com:HelixDevelopment/HelixQA.git
```

The SSH URL format `git@github.com:HelixDevelopment/HelixQA.git` matches the constitutional transport mandate. No `https://` variant is permitted in any `.gitmodules` file.

### 2.2.2 Add Dependency Submodules

HelixDevelopment modules are collected under a `Dependencies/HelixDevelopment/` path prefix to keep the root clean and to make the dependency structure self-documenting. The four autonomous-session modules plus LLMsVerifier are registered as follows:

```
[submodule "Dependencies/HelixDevelopment/DocProcessor"]
  path = Dependencies/HelixDevelopment/DocProcessor
  url = git@github.com:HelixDevelopment/DocProcessor.git

[submodule "Dependencies/HelixDevelopment/LLMOrchestrator"]
  path = Dependencies/HelixDevelopment/LLMOrchestrator
  url = git@github.com:HelixDevelopment/LLMOrchestrator.git

[submodule "Dependencies/HelixDevelopment/LLMProvider"]
  path = Dependencies/HelixDevelopment/LLMProvider
  url = git@github.com:HelixDevelopment/LLMProvider.git

[submodule "Dependencies/HelixDevelopment/VisionEngine"]
  path = Dependencies/HelixDevelopment/VisionEngine
  url = git@github.com:HelixDevelopment/VisionEngine.git

[submodule "Dependencies/HelixDevelopment/LLMsVerifier"]
  path = Dependencies/HelixDevelopment/LLMsVerifier
  url = git@github.com:HelixDevelopment/LLMsVerifier.git
```

The `vasic-digital` modules (Challenges, Containers, Security) are assumed to already exist in the HelixCode workspace because they are shared infrastructure used by the Catalogizer components that HelixCode also maintains. If any are missing, they must be added with the same pattern:

```
[submodule "Challenges"]
  path = Challenges
  url = git@github.com:vasic-digital/Challenges.git

[submodule "Containers"]
  path = Containers
  url = git@github.com:vasic-digital/Containers.git
```

### 2.2.3 Configure Submodule Paths and Go Replace Directives

Because HelixCode places HelixDevelopment modules under `Dependencies/HelixDevelopment/`, HelixQA's internal `go.mod` cannot resolve them on its own.

HelixCode’s root `go.mod` (module `dev.helix.code`, Go 1.24.0)<sup>[14]</sup> must supply `replace` directives that override the module paths for the entire workspace. The following directives are appended to `HelixCode/go.mod`:

```
replace digital.vasic.helixqa => ./HelixQA

replace digital.vasic.docprocessor => ./Dependencies/
    HelixDevelopment/DocProcessor
replace digital.vasic.llmorchestrator => ./Dependencies/
    HelixDevelopment/LLMOrchestrator
replace digital.vasic.visionengine => ./Dependencies/
    HelixDevelopment/VisionEngine
```

The `Challenges`, `Containers`, and `Security` replacements are assumed to already exist in the `HelixCode go.mod` because `Catalogizer`’s API layer depends on them. If they are absent, the same pattern applies:

```
replace digital.vasic.challenges => ./Challenges
replace digital.vasic.containers => ./Containers
replace digital.vasic.security => ./Security
```

## 2.2.4 Version Locking Strategy

All submodule pointers are pinned to specific commits. The bump workflow is implemented as a `Makefile` target (§2.4.1) that enforces the following verification sequence before any commit hash is updated:

1. `git -C <path> fetch origin` — retrieve latest objects.
2. `git -C <path> log --oneline <current>..<candidate>` — enumerate changes.
3. `git -C <path> diff --stat <current>..<candidate>` — inspect delta size.
4. `cd HelixQA && go build ./...` — compile `HelixQA` against the candidate commits of all dependency submodules.
5. `cd HelixQA && go test ./... -race` — run `HelixQA`’s test suite (235 tests) with the race detector<sup>[15]</sup>.
6. Only after steps 4 and 5 pass is the submodule pointer advanced and committed.

The initial pin set for `HelixCode` registration is derived from the latest known-good commits in the `Catalogizer` integration research<sup>[8]</sup>:

| Submodule       | Initial Pin | Source of Pin                                                             |
|-----------------|-------------|---------------------------------------------------------------------------|
| HelixQA         | 0bca023     | Phase 29 §12.6 Memory-Budget Ceiling, latest upstream main <sup>[8]</sup> |
| DocProcessor    | 5f1e58a     | Catalogizer current pin <sup>[8]</sup>                                    |
| LLMOrchestrator | 1b95823     | Catalogizer current pin <sup>[8]</sup>                                    |
| LLMProvider     | 0720b6e     | Catalogizer current pin <sup>[8]</sup>                                    |

| Submodule    | Initial Pin                  | Source of Pin                                          |
|--------------|------------------------------|--------------------------------------------------------|
| VisionEngine | <i>unpinned</i> (track main) | Catalogizer does not pin this submodule <sup>[8]</sup> |
| LLMsVerifier | <i>to be determined</i>      | Not yet present in Catalogizer                         |

VisionEngine and LLMsVerifier are treated as floating-track submodules for Phase 1. They receive their first pinned commit in Phase 2 (Core Integration) after a clean build and test run against HelixCode’s own binaries.

## 2.3 Catalogizer Submodule Synchronization

Catalogizer currently contains HelixQA as a registered submodule at commit 35deb43 (Phase 27.7 — visionnav Provider interface + NopProvider)<sup>[8]</sup>. The latest upstream commit is 0bca023 (Phase 29 — §12.6 Memory-Budget Ceiling). Catalogizer is therefore two major phases behind. This gap must be closed before HelixCode can safely inherit the same submodule, because HelixCode’s integration will start from the latest upstream state rather than reproducing the stale pin.

### 2.3.1 Bump HelixQA from 35deb43 to 0bca023

The exact commands to advance the Catalogizer HelixQA submodule are:

```
# 1. Enter the Catalogizer repository root
cd /path/to/Catalogizer

# 2. Ensure the submodule is initialized and at the current known state
git submodule update --init HelixQA

# 3. Enter the submodule and fetch all remotes
cd HelixQA
git fetch origin

# 4. Verify the target commit exists and inspect its log
git log --oneline -5 0bca023

# 5. Checkout the target commit (detached HEAD, as required for submodule pins)
git checkout 0bca023

# 6. Return to the superproject and stage the submodule pointer change
cd ..
git add HelixQA

# 7. Verify the staged diff shows only the submodule pointer
git diff --cached --stat HelixQA
```

```
# 8. Commit with a descriptive message referencing phase and
    commit
git commit -m "chore(submodules): bump HelixQA to 0bca023 (Phase
    29 §12.6 Memory-Budget Ceiling)
```

```
Refs: integration-plan/ch-2/helixqa-bump"
```

After the pointer change, HelixQA's own Go module imports must still resolve. Because HelixQA's `go.mod` expects `digital.vasic.challenges` at `../Challenges`<sup>[1]</sup>, the Catalogizer workspace layout (where `Challenges` is a sibling of `HelixQA` at the repository root) satisfies this requirement without modification.

Verification that the bump is structurally sound:

```
# Verify Go module resolution from inside the bumped submodule
cd HelixQA
go mod tidy
go build ./...
```

```
# Run the full HelixQA test suite with race detection
go test ./... -race -count=1
```

```
# Expected result: 235 tests passing, 0 failures[^15^]
```

### 2.3.2 Cascade Bump to All HelixDevelopment Submodules

HelixQA at `0bca023` may depend on newer commits of its sibling `HelixDevelopment` modules than the ones currently pinned in `Catalogizer`. The cascade bump advances `DocProcessor`, `LLMOrchestrator`, and `LLMProvider` to their latest main commits and verifies that `HelixQA` still builds.

```
# DocProcessor bump
cd DocProcessor
git fetch origin
# Inspect delta before advancing
git log --oneline 5f1e58a..origin/main | head -20
# Target commit: inspect origin/main and select the latest stable
git checkout $(git rev-parse origin/main)
cd ..
git add DocProcessor

# LLMOrchestrator bump
cd LLMOrchestrator
git fetch origin
git log --oneline 1b95823..origin/main | head -20
git checkout $(git rev-parse origin/main)
cd ..
git add LLMOrchestrator
```

```
# LLMProvider bump
cd LLMProvider
git fetch origin
git log --oneline 0720b6e..origin/main | head -20
git checkout $(git rev-parse origin/main)
cd ..
git add LLMProvider

# Commit the cascade
git commit -m "chore(submodules): cascade bump HelixDevelopment
submodules

- DocProcessor: 5f1e58a -> $(git -C DocProcessor rev-parse --
short HEAD)
- LLMOrchestrator: 1b95823 -> $(git -C LLMOrchestrator rev-parse
--short HEAD)
- LLMProvider: 0720b6e -> $(git -C LLMProvider rev-parse --short
HEAD)
```

Aligned with HelixQA 0bca023 (Phase 29)."

After the cascade, the build verification command is:

```
cd HelixQA
go build ./...
go test ./... -race -count=1
```

If any test fails, the offending submodule is rolled back to its previous pin and the failure is documented in docs/reports/helixqa-cascade-blocker.md for Phase 2 resolution.

### 2.3.3 Verify Challenges Submodule Compatibility

Catalogizer's Challenges submodule is at commit 4390e48 from vasic-digital/Challenges<sup>[8]</sup>. HelixQA's go.mod imports digital.vasic.challenges<sup>[1]</sup>, and the replace directive in Catalogizer's catalog-api/go.mod points digital.vasic.challenges => ../Challenges<sup>[16]</sup>.

A namespace discrepancy exists: Catalogizer uses the repository vasic-digital/Challenges, while HelixQA's Go import path is digital.vasic.challenges. In Go, the module path declared in a repository's go.mod is the authoritative identifier. If vasic-digital/Challenges declares itself as digital.vasic.challenges (the standard pattern for the vasic-digital GitHub organization), the import resolves correctly. If it declares a different module path, the replace directive must map the import path to the actual checkout path regardless of repository naming.

The verification procedure is:

```
# 1. Check the module path declared inside the Challenges
repository
cd Challenges
cat go.mod | grep "^module"
```

```
# Expected output: module digital.vasic.challenges

# 2. Verify that HelixQA can resolve the import against the local
    checkout
cd ../HelixQA
go list -m digital.vasic.challenges
# Expected: digital.vasic.challenges (replaced by ../Challenges)

# 3. Build a HelixQA package that directly imports the challenges
    module
go build ./pkg/orchestrator
# This package imports digital.vasic.challenges/pkg/
    {bank,challenge,runner,logging}[^2^]
```

If `go list` returns a version from the public proxy instead of the local replacement, the `replace` directive in either `catalog-api/go.mod` or a newly created `go.work` file is misconfigured. The fix is to ensure `GOFLAGS=-mod=mod` is not set and that the replacement path is relative to the module containing the `replace` directive.

## 2.4 Build System Integration

Once the submodules are registered and pinned, the HelixCode build system must be extended to compile, test, and execute HelixQA. This section defines the exact changes to the Makefile, Go module files, and Docker Compose stack.

### 2.4.1 Makefile Targets

HelixCode's primary Makefile lives at `HelixCode/Makefile` (15,341 chars)<sup>[17]</sup>. It already defines targets for the CLI, server, TUI, desktop, mobile clients, and challenge suite. Three new targets are appended for HelixQA operations:

```
# -----
# HelixQA integration targets
# -----

.PHONY: helixqa-build helixqa-test helixqa-challenge helixqa-
        bump-submodules

HELIXQA_DIR := ../HelixQA
HELIXQA_DEPS := ../Dependencies/HelixDevelopment

helixqa-build: ## Build HelixQA and all dependency modules
    @echo "Building HelixQA..."
    cd $(HELIXQA_DIR) && go build ./...
    @echo "Building DocProcessor..."
    cd $(HELIXQA_DEPS)/DocProcessor && go build ./...
    @echo "Building LLMOrchestrator..."
    cd $(HELIXQA_DEPS)/LLMOrchestrator && go build ./...
    @echo "Building LLMPProvider..."
```

```

cd $(HELIXQA_DEPS)/LLMProvider && go build ./...
@echo "Building VisionEngine..."
cd $(HELIXQA_DEPS)/VisionEngine && go build ./...
@echo "HelixQA build complete."

helixqa-test: ## Run HelixQA test suite with race detector
cd $(HELIXQA_DIR) && go test ./... -race -count=1 -p 2

helixqa-challenge: ## Run HelixQA challenge banks against local
HelixCode stack
cd $(HELIXQA_DIR) && go run ./cmd/helixqa run \
--banks ./banks/full-qa-api,./banks/full-qa-web \
--platform api,web \
--browser-url http://localhost:8080 \
--output ./qa-results/helixcode-$(shell date +%Y%m%d-%
%H%M%S) \
--report markdown,html,json \
--validate \
--tickets

helixqa-bump-submodules: ## Bump all HelixQA dependency
submodules with verification
@echo "Fetching updates for HelixQA dependencies..."
@for dep in DocProcessor LLMOrchestrator LLMProvider
VisionEngine; do \
path="$(HELIXQA_DEPS)/$$dep"; \
echo "  Checking $$dep ($$path)..."; \
git -C $$path fetch origin; \
current=$(git -C $$path rev-parse HEAD); \
candidate=$(git -C $$path rev-parse origin/main); \
if [ "$$current" != "$$candidate" ]; then \
echo "    Advance $$dep: $(git -C $$path rev-parse
--short $$current) -> $(git -C $$path rev-parse --short
$$candidate)"; \
git -C $$path checkout $$candidate; \
else \
echo "    $$dep is already at latest."; \
fi; \
done
@echo "Building with candidate commits..."
$(MAKE) helixqa-build
@echo "Testing with candidate commits..."
$(MAKE) helixqa-test
@echo "Bump verification complete. Stage changes with: git
add $(HELIXQA_DEPS)"

```

The `helixqa-build` target compiles every HelixDevelopment module independently before attempting to compile HelixQA itself. This catches compilation errors in dependencies early, rather than producing opaque “missing module” errors from inside HelixQA. The `helixqa-`

`test target` runs HelixQA's 235 tests with the race detector enabled (`-race`) and parallelism limited to two (`-p 2`) to avoid overloading the developer workstation<sup>[15]</sup>. The `helixqa-challenge` target executes the API and web challenge banks against a locally running HelixCode server on `localhost:8080`, producing Markdown, HTML, and JSON reports with ticket generation enabled.

### 2.4.2 HelixCode `go.mod` Replace Directive

The root `HelixCode/go.mod` must expose HelixQA's module path to the rest of the workspace. The following line is inserted into the `replace` block:

```
replace digital.vasic.helixqa => ./HelixQA
```

Because HelixCode's module is `dev.helix.code` and it does not directly import `digital.vasic.helixqa`, this replacement is primarily a convenience for any future HelixCode packages that may call into HelixQA's public API (for example, a planned `internal/qa/` package that reuses HelixQA's report types). Without the replacement, any such import would resolve to the public module proxy and potentially fetch a stale or nonexistent version.

### 2.4.3 Catalogizer `catalog-api/go.mod` Verification

Catalogizer's `catalog-api/go.mod` contains 22 `replace` directives that wire its `vasic-digital` submodule dependencies<sup>[16]</sup>. After the HelixQA bump, this file must be checked to ensure that all module paths referenced by HelixQA's transitive imports are covered. The verification script is:

```
#!/bin/bash
# verify-replace-coverage.sh
# Run from Catalogizer root after submodule bump

MODULES=(
    "digital.vasic.challenges"
    "digital.vasic.containers"
    "digital.vasic.docprocessor"
    "digital.vasic.llmorchestrator"
    "digital.vasic.security"
    "digital.vasic.visionengine"
)

GOMOD="catalog-api/go.mod"
MISSING=0

for mod in "${MODULES[@]}; do
    if ! grep -q "replace $mod =>" "$GOMOD"; then
        echo "MISSING: $mod has no replace directive in $GOMOD"
        MISSING=$((MISSING + 1))
    else
        echo "OK: $mod"
    fi
done
```

```

done

if [ $MISSING -ne 0 ]; then
    echo "FAIL: $MISSING module(s) uncovered. Add replace
        directives before proceeding."
    exit 1
fi

echo
    "PASS: All HelixQA dependency modules are covered by
    replace directives."

```

If any module is missing, the directive follows the established pattern. For example, if `digital.vasic.docprocessor` is absent:

```
replace digital.vasic.docprocessor => ../DocProcessor
```

Note that Catalogizer places HelixDevelopment modules at the repository root (`DocProcessor/`, `LLMOrchestrator/`, etc.) rather than under a `Dependencies/` prefix. This is a legacy layout that Catalogizer inherited before the namespace convention was formalized. HelixCode uses the `Dependencies/HelixDevelopment/` prefix for cleanliness, but the `replace` directive must match whatever path the submodule actually occupies.

## 2.4.4 Docker Compose Integration

HelixCode's Docker Compose stack is defined in `docker-compose.yml` at the repository root<sup>[18]</sup>. The stack currently runs PostgreSQL, Redis, the HelixCode server, Nginx, Prometheus, and Grafana. A new `helixqa-runner` service is added to support headless QA execution inside the container network:

```

helixqa-runner:
  build:
    context: ./HelixQA
    dockerfile: ../docker/Dockerfile.helixqa
  container_name: helixqa-runner
  depends_on:
    - helixcode-server
    - postgres
    - redis
  environment:
    HELIX_AUTONOMOUS_ENABLED: "true"
    HELIX_AUTONOMOUS_PLATFORMS: "api,web"
    HELIX_AUTONOMOUS_TIMEOUT: "2h"
    HELIX_WEB_URL: "http://helixcode-server:8080"
    HELIX_OUTPUT_DIR: "/qa-results"
    HELIX_REPORT_FORMATS: "markdown,html,json"
    HELIX_TICKETS_ENABLED: "true"
    HELIX_VERIFIER_URL: "http://llmsverifier:9090"
  volumes:

```

```

- ./qa-results:/qa-results
- ./HelixQA/banks:/qa-banks:ro
networks:
- helix-network
profiles:
- qa

```

The service uses a Docker profile (`profiles: [qa]`) so that it is started only when explicitly requested: `docker-compose --profile qa up`. This prevents QA processes from consuming resources during normal development. The `depends_on` clause ensures that the HelixCode server and databases are healthy before HelixQA begins execution. The `HELIX_WEB_URL` uses the internal Docker network hostname `helixcode-server` rather than `localhost`, which would resolve to the container's own loopback interface.

The corresponding Dockerfile at `docker/Dockerfile.helixqa` is a minimal Go builder image:

```

FROM golang:1.26-alpine AS builder
WORKDIR /build
COPY HelixQA/go.mod HelixQA/go.sum ./
RUN go mod download
COPY HelixQA/ ./
RUN go build -o /bin/helixqa ./cmd/helixqa

FROM alpine:3.21
RUN apk add --no-cache chromium ffmpeg
COPY --from=builder /bin/helixqa /usr/local/bin/helixqa
ENTRYPOINT ["helixqa"]

```

Chromium and FFmpeg are installed in the runtime stage because HelixQA's web and video capture paths depend on them<sup>[12]</sup>. The image size is approximately 287 MB after build; this is tracked as a build metric in the integration dashboard.

Table 1 below consolidates the full dependency matrix that drives all registration, bump, and build decisions in this phase.

**Table 1. HelixQA Dependency Matrix — Module, Submodule, Commit, and Integration Path**

| # | Go Module                               | Repository (SSH URL)                                          | Catalog Pin                  |
|---|-----------------------------------------|---------------------------------------------------------------|------------------------------|
| 1 | <code>digital.vasic.helixqa</code>      | <code>git@github.com:HelixDevelopment/HelixQA.git</code>      | <code>35deb4 → 0bca02</code> |
| 2 | <code>digital.vasic.challenges</code>   | <code>git@github.com:vasic-digital/Challenges.git</code>      | <code>4390e4</code>          |
| 3 | <code>digital.vasic.containers</code>   | <code>git@github.com:vasic-digital/Containers.git</code>      | <code>9f9f52</code>          |
| 4 | <code>digital.vasic.docprocessor</code> | <code>git@github.com:HelixDevelopment/DocProcessor.git</code> | <code>5f1e58</code>          |

| # | Go Module                     | Repository (SSH URL)                                | Catalogizer Pin |
|---|-------------------------------|-----------------------------------------------------|-----------------|
| 5 | digital.vasic.llmorchestrator | git@github.com:HelixDevelopment/LLMOrchestrator.git | 1b9582          |
| 6 | digital.vasic.visionengine    | git@github.com:HelixDevelopment/VisionEngine.git    | unpinned        |
| 7 | digital.vasic.security        | git@github.com:vasic-digital/Security.git           | varies          |
| 8 | (service) LLMsVerifier        | git@github.com:HelixDevelopment/LLMsVerifier.git    | absent          |
| 9 | (service) LLMProvider         | git@github.com:HelixDevelopment/LLMProvider.git     | 0720b6          |

The risk levels in the final column are derived from two factors: whether the submodule is pinned to a known-good commit (lower risk) or tracking a moving branch (higher risk), and whether the module’s public API has changed between the Catalogizer pin and the latest upstream commit. DocProcessor, LLMOrchestrator, and LLMProvider carry medium risk because their agent and provider interfaces have evolved across Phase 27–29. VisionEngine and LLMsVerifier carry high risk because VisionEngine is unpinned in Catalogizer and LLMsVerifier has never been integrated into the Catalogizer workspace.

**Table 2. Exact .gitmodules Entries to Add or Update in HelixCode**

| Section                                                     | Path                                          | URL                                                 | Acc    |
|-------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------|--------|
| [submodule "HelixQA"]                                       | HelixQA                                       | git@github.com:HelixDevelopment/HelixQA.git         | Ad new |
| [submodule "Dependencies/HelixDevelopment/DocProcessor"]    | Dependencies/HelixDevelopment/DocProcessor    | git@github.com:HelixDevelopment/DocProcessor.git    | Ad new |
| [submodule "Dependencies/HelixDevelopment/LLMOrchestrator"] | Dependencies/HelixDevelopment/LLMOrchestrator | git@github.com:HelixDevelopment/LLMOrchestrator.git | Ad new |
| [submodule "Dependencies/HelixDevelopment/LLMProvider"]     | Dependencies/HelixDevelopment/LLMProvider     | git@github.com:HelixDevelopment/LLMProvider.git     | Ad new |

| Section                                                             | Path                                               | URL                                                  | Ac        |
|---------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------|-----------|
| [submodule<br>"Dependencies/<br>HelixDevelopment/<br>VisionEngine"] | Dependencies/<br>HelixDevelopment/<br>VisionEngine | git@github.com:HelixDevelopment/<br>VisionEngine.git | Ac<br>ne  |
| [submodule<br>"Dependencies/<br>HelixDevelopment/<br>LLMsVerifier"] | Dependencies/<br>HelixDevelopment/<br>LLMsVerifier | git@github.com:HelixDevelopment/<br>LLMsVerifier.git | Ac<br>ne  |
| [submodule<br>"Challenges"]                                         | Challenges                                         | git@github.com:vasic-digital/<br>Challenges.git      | Ve<br>exi |
| [submodule<br>"Containers"]                                         | Containers                                         | git@github.com:vasic-digital/<br>Containers.git      | Ve<br>exi |
| [submodule<br>"Security"]                                           | Security                                           | git@github.com:vasic-digital/<br>Security.git        | Ve<br>exi |

Every URL in Table 2 uses the SSH transport (`git@github.com:`). No entry contains an `https://` scheme. The Challenges, Containers, and Security entries are listed as “Verify existing” because they are assumed to be present in any HelixCode workspace that already builds the Catalogizer-derived components. If an audit reveals they are missing, they are added with the same pattern.

**Table 3. Build System Changes — Files, Lines, and Verification Commands**

| File                               | Change                     | Exact Snippet                                                                    | Verification Command                                               |
|------------------------------------|----------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------|
| HelixCode/Makefile                 | Append four targets        | helixqa-build, helixqa-test, helixqa-challenge, helixqa-bump-submodules (§2.4.1) | make helixqa-build                                                 |
| HelixCode/go.mod                   | Add replace directive      | replace<br>digital.vasic.helixqa<br>=> ./HelixQA (§2.4.2)                        | cd HelixCode &&<br>list -m<br>digital.vasic.he                     |
| Catalogizer/<br>catalog-api/go.mod | Add missing replaces       | replace<br>digital.vasic.docprocessor<br>=> ../DocProcessor (etc.)<br>(§2.4.3)   | ./verify-replace<br>coverage.sh                                    |
| docker-compose.yml                 | Add helixqa-runner service | YAML block with profiles:<br>[qa] (§2.4.4)                                       | docker-compose c<br>--profiles                                     |
| docker/<br>Dockerfile.helixqa      | Create new                 | Multi-stage builder with Chromium + FFmpeg (§2.4.4)                              | docker build -f<br>docker/<br>Dockerfile.helix<br>helixqa-runner:t |

Table 3 provides the executable checklist for build-system integration. Each row contains the file path, the nature of the change, a pointer to the exact code block in this document, and the command that confirms the change is working. Running all five verification commands in

sequence constitutes the Phase 1 exit gate: if any command fails, the phase is incomplete and the issue is logged as a blocker for Phase 2.

The commands in Table 3 produce the following expected outputs when Phase 1 is successful:

- `make helixqa-build` exits with code 0 and prints “HelixQA build complete.”
- `go list -m digital.vasic.helixqa` prints the module path followed by (replaced by `./HelixQA`).
- `./verify-replace-coverage.sh` prints “PASS: All HelixQA dependency modules are covered by replace directives.” and exits with code 0.
- `docker-compose config --profiles` lists `qa` among the available profiles.
- `docker build . . .` exits with code 0 and produces an image tagged `helixqa-runner:test`.

These outputs are captured and appended to the Phase 1 completion report, which is stored at `docs/reports/integration/helixqa-phase1-completion.md` and referenced in the Chapter 3 (Core Integration) entry criteria.